

3.Action Bridge 实现（2026-03-24）



本次会话目标：将 trajectory_bridge 从订阅 controller_state 话题改为实现 FollowJointTrajectory Action Server，让 MoveIt 直接与 bridge 通信，获得完整的执行反馈。

1. 问题背景

原架构的 trajectory_bridge 通过订阅 /arm_controller/controller_state 和 /hand_controller/controller_state 话题获取关节目标位置，再转发给 micro-ROS（ESP32-S3）。这种方式存在以下缺陷：

- MoveIt 无法知道机器人是否真正执行到位，缺少反馈闭环
- JointTrajectoryController 通过 Action 接口执行轨迹，直接读 controller_state 存在时序不稳定问题
- MoveIt 不知道 bridge 的执行状态，规划器无法正确判断执行成功与否

2. 新架构设计

```
MoveIt (OMPL 规划)
  ↓ FollowJointTrajectory Action Goal
trajectory_bridge (Action Server × 2)
  ↓ 按时间点执行, rad → deg
/servo_commands (Float32MultiArray, 6个角度)
  ↓
micro-ROS Agent → ESP32-S3 → PCA9685 → 舵机
  ↓ Feedback / Result 回传
MoveIt (获知执行状态, 闭环完成)
```

3. 修改的文件

3.1 trajectory_bridge_node.cpp（完全重写）

- 创建两个 FollowJointTrajectory Action Server：
 - arm_controller/follow_joint_trajectory (joint_1~5)
 - hand_controller/follow_joint_trajectory (joint_6)
- 轨迹执行在独立线程中运行 (handle_accepted → detach thread)
- 按轨迹点的 time_from_start 精确等待后发布 /servo_commands (rad → deg)
- 支持取消请求 (is_canceling() 检查)
- 每个轨迹点执行后向 MoveIt 发送 Feedback，全部完成后返回 SUCCESSFUL Result

- 使用 `jointmap + mutex` 合并 `arm` 和 `hand` 的关节，保证 6 个关节按固定顺序 (`joint_1~6`) 输出

3.2 CMakeLists.txt

- 新增 `find_package(rclcpp_action REQUIRED)`
- 在 `ament_target_dependencies` 中加入 `rclcpp_action`

3.3 package.xml

- 新增 `rclcpp_action`
- 版本升至 2.0.0，描述更新为 Action Server 架构

4. 编译命令

```
cd ~/Desktop/ros2_ws
colcon build --packages-select trajectory_bridge
```

5. moveit_controllers.yaml 配置确认

无需修改，原有配置已正确指向 Action 接口：

```
moveit_simple_controller_manager:
  arm_controller:
    action_ns: follow_joint_trajectory # ✅ 与 bridge 的 action server 名一致
    type: FollowJointTrajectory
    joints: [joint_1, joint_2, joint_3, joint_4, joint_5]
  hand_controller:
    action_ns: follow_joint_trajectory
    type: FollowJointTrajectory
    joints: [joint_6]
```

6. Notion MCP 配置记录

- 使用 `@notionhq/notion-mcp-server` 配置 Claude Code MCP 连接 Notion
- Token 环境变量格式：`OPENAPI_MCP_HEADERS` (含 `Authorization + Notion-Version`)
- 保存为用户级全局配置 (`-s user`)，所有项目均可使用
- ⚠️ 注意：每个 Notion 页面需要手动添加集成连接，或在工作区 `Settings → Connections` 中全局启用



待办：在终端运行 `colcon build --packages-select trajectory_bridge` 验证编译通过，然后启动 `bridge` 节点测试 MoveIt 规划执行是否有正确反馈。

7. 实测结果



完整链路测试通过！MoveIt → Action Bridge → `/servo_commands` → micro-ROS 全部连通。

7.1 Bridge 节点日志

```
[INFO] TrajectoryBridge Action Server 已启动 - 关节数：6，发布频率：50.0 Hz
[INFO] 收到轨迹目标，共 165 个点
[INFO] 开始执行轨迹，共 165 个点
[INFO] 轨迹执行完成（165 个点）
```

执行时长：约 16.4 秒（时间戳 1774329991 → 1774330007）

7.2 各节点验证状态

- ✅ MoveIt → Bridge：FollowJointTrajectory Action Goal 正常接收（165 个轨迹点）
- ✅ Bridge 执行：按时间点逐个执行，16.4 秒全部完成
- ✅ Result 回传：Bridge 返回 SUCCESSFUL，MoveIt 确认执行完成，闭环建立
- ✅ `/servo_commands` 有输出：Float32MultiArray（6 个角度，单位°）正常发布给 micro-ROS ESP32-S3

7.3 完整验证数据流

RViz 拖动目标



MoveIt (OMPL 规划) → FollowJointTrajectory Action Goal (165点) 



trajectory_bridge Action Server 



`/servo_commands` (Float32MultiArray, 6个角度, rad→deg) 



micro-ROS ESP32-S3 → PCA9685 → 舵机 



Result: SUCCESSFUL → 回传给 MoveIt 



测试时间：2026-03-24

环境：ROS2 Humble + micro-ROS + ESP32-S3 + PCA9685